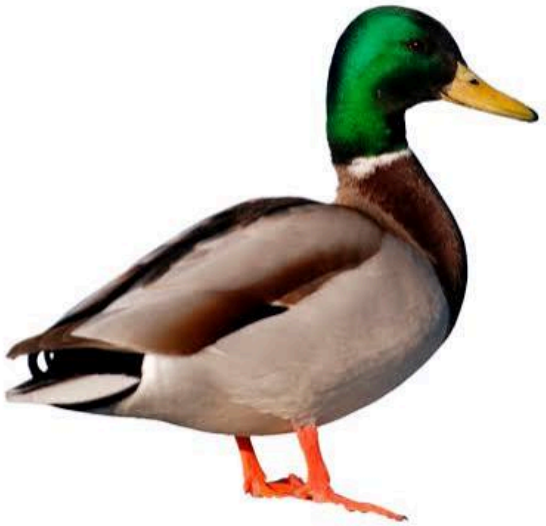
Ciência de Dados Aplicada ao Direito I

Aula 20

Uma análise do início ao fim



Prof. Bruno da Cunha de Oliveira - 2026/2

Gramática e vocabulário

Insper

Até aqui

Variáveis **Condicionais** **Listas** **Loops**

Dicionários **Funções** **Objetos**

Mas a gramática não é suficiente

Deblaterar

Deblaterar

falar com veemência contra alguém ou algo; bradar

Estugar

Estugar

apressar o passo; estimular, incitar

Bibliotecas aumentam o vocabulário

Vocês sabem a gramática do português, e mesmo assim não sabiam essas palavras. No Python é igual: a gramática vocês têm, e cada biblioteca traz palavras novas, prontas para importar e usar.

`jus.scraper`, `cjsg`, `raspe.camara` foram as primeiras. Quem sabe as palavras consegue dizer à IA, com precisão, o que quer.

Cada área tem o seu vocabulário

Área	Palavras que quem é de fora não usa
Direito	preclusão, litispendência, agravo interno
Medicina	anamnese, etiologia, prognóstico
Computação	compilador, cache, concorrência

Ninguém aprende todos. Cada um aprende o da área em que trabalha, e as bibliotecas também se dividem por área.

Os dois vocabulários do resto do curso

Bibliotecas	Para quê
para usar IA: LangChain	pedir a um modelo de linguagem, de dentro do programa, que leia, classifique e extraia
para bases de dados: pandas e as de gráfico	ler, filtrar, juntar e resumir tabelas, e desenhar o resultado

Em CDAD 2: conceitos de estatística e as bibliotecas de estatística.

Um cliente, três perguntas

Insper

Uma empresa quer entrar no Brasil

Uma empresa vende produtos médicos à base de canabidiol e quer começar a vender no Brasil. Ela sabe que boa parte do acesso a medicamentos aqui passa pela Justiça, e quer entender isso antes de decidir.

#	O que ela quer saber
1	As pessoas judicializam mais contra entes públicos ou contra privados?
2	Dentro de cada grupo, quem é mais acionado?
3	Quem ganha e quem perde?

Como vocês fariam?

Com o que vocês já sabem (o juscraper, objetos, listas, `for`), como tentariam responder às três perguntas?

Escrevam individualmente o seu palpite no formulário, com o máximo de detalhamento possível: de onde viriam os dados, o que faria com eles e quais dificuldades antecipam.

O caminho de hoje

RESPOSTA

Etapa	Hoje
coletar	sentenças do TJSP, já num CSV
abrir e olhar	pandas: uma tabela, colunas
extrair	o réu e o resultado, do texto
contar	público ou privado; quem ganha

Cada etapa ganha aulas próprias até o fim do curso. Hoje é a versão express.

A base de hoje

O que está no CSV do Blackboard

BUSCA

canabidiol

no banco de sentenças do TJSP, sem outro filtro

RESULTADOS

4.618

sentenças, em 462 páginas

NO CSV

300

30 páginas sorteadas, 10 por página

Coletada com o juscraper. Uma sentença por linha: classe, assunto, comarca, data e o texto inteiro.

O que o termo decide

A base veio da busca por "canabidiol". Se eu tivesse buscado "canabidiol OU CBD OU cannabis", ou filtrado só a classe "Mandado de Segurança", ou só 2025, ou o TJRJ, o que mudaria na resposta ao cliente?

O resultado vem do mundo ou da coleta?

RESPOSTA

O que aparece como "quem é mais acionado" pode ser um fato do mundo ou um efeito do que eu decidi coletar: o termo, a classe, o período, o tribunal e a instância.

Antes de responder ao cliente, cada uma dessas escolhas precisa estar escrita e justificada. Como escolher bem é a próxima aula. Hoje a base é esta.

Abrir a base

Abrir o CSV

Baixem `decisoes-canabidiol-tjsp.csv` do Blackboard para a pasta `cdad1` e, no caderno:

```
import pandas as pd

decisoes = pd.read_csv("decisoes-canabidiol-tjsp.csv")
print(len(decisoes))
print(decisoes.columns)
```

O que cada `print` mostra?

300 linhas, cinco colunas

RESPOSTA

```
300
Index(['classe', 'assunto', 'comarca', 'data', 'texto'],
dtype='str')
```

`read_csv` lê o arquivo e devolve um DataFrame: uma tabela, com linhas e colunas. `len` conta as linhas; `.columns` lista os nomes das colunas.

Onde está o que o cliente quer?

As colunas são `classe`, `assunto`, `comarca`, `data` e `texto`. Qual delas diz quem é o réu? E qual diz quem ganhou?

Nenhuma coluna pronta: está no texto

RESPOSTA

Nenhuma. O banco de sentenças do TJSP não entrega as partes nem o resultado como coluna. As duas informações estão dentro de `texto`, e o trabalho de hoje é tirá-las de lá e pô-las em colunas novas.

Uma coluna se acessa pelo nome entre colchetes: `decisoes["texto"]`.

Só o começo de cada texto

Cada sentença tem páginas. Para olhar várias de uma vez, basta o começo. Em Python, um pedaço de um texto se pega com colchetes e dois-pontos:

```
texto = "SENTENÇA\n\nProcesso Digital nº: 1500375-25.2024..."
print(texto[:8])
print(texto[:20])
```

O que cada `print` mostra?

Fatia: do começo até a posição que vocês pedem

RESPOSTA

```
SENTENÇA
SENTENÇA

Processo D
```

`[:8]` é a fatia do texto do começo até o caractere 8, sem ele; `[:20]`, até o 20. Vale para texto e para lista: `[:5]` numa coluna são as cinco primeiras linhas. A contagem começa no zero.

Criar uma coluna nova

```
import pandas as pd

decisoes = pd.read_csv("decisoes-canabidiol-tjsp.csv")
decisoes["inicio"] = decisoes["texto"].str[:200]
print(decisoes.columns)
print(decisoes["inicio"][7])
```

Seis colunas, e o começo da linha 7

```
Index(['classe', 'assunto', 'comarca', 'data', 'texto',
'inicio'], dtype='str')
SENTENÇA

Processo nº:  0004177-52.2023.8.26.0664
Classe - Assunto  Procedimento do Juizado Especial da Fazenda
Pública - Fornecimento de medicamentos
Requerente: Maria José Caetano
Requerido:  Fazenda P
```

O que a linha nova fez

Pedaço	O que é
`decisoes["inicio"] = ...`	coluna nova, criada ao receber um valor
`decisoes["texto"].str`	as operações de texto, aplicadas a cada linha da coluna
`[:200]`	a fatia: os 200 primeiros caracteres de cada texto
`decisoes["inicio"][7]`	a coluna, e dentro dela a linha 7 (a oitava: a contagem começa no zero)

Ler cinco decisões

Insper

Os cinco primeiros textos, o começo de cada um

```
import pandas as pd

decisoes = pd.read_csv("decisoes-canabidiol-tjsp.csv")
for texto in decisoes["texto"][:5]:
    print(texto[:200])
    print("----------")
```

`[:5]` pega as cinco primeiras linhas da coluna; `[:200]`, o começo de cada texto. Leiam o que sai.

A primeira

```
SENTENÇA
Processo Digital nº:  1500375-25.2024.8.26.0344
Classe - Assunto  Petição Cível - Petição intermediária
Requerente: Braz Bento dos Santos
Requerido:  Fazenda Pública do Estado de São Paulo e ou
```

A segunda, igual

```
SENTENÇA
Processo Digital nº:  1052225-21.2022.8.26.0224
Classe - Assunto  Procedimento Comum Cível - Tratamento
médico-hospitalar
Requerente: Enzo Henrique Rodrigues Chagas
Requerido:  UNIMED DE GUARUL
```

A terceira não começa assim

```
TERMO DE CONCLUSÃO
Eu,           JHENYFFER SANTINA MARIA REIS MORES, Estagiário
Nível Superior, matr. nº E50318838, em 26 de fevereiro de
2024, faço estes autos conclusos ao(à) MM. Juiz(a) de Direito
```

Nem a quarta

```
SENTENÇA

Processo nº:  1019802-69.2023.8.26.0451 - 2023/001900

Vistos.

Matheus Mondoni de Oliveira, ajuizou Ação Procedimento Comum
Cível contra Unimed de Piracicaba alegando, em síntese,ser ade
```

Que padrão vocês veem?

Nas duas que começam pelo cabeçalho, onde está o réu? E quem pede?

Se bastasse procurar esse padrão em cada texto, o que vocês coletariam de cada sentença?

Requerente e Requerido

RESPOSTA

O cabeçalho traz `Requerente:` e `Requerido:`, cada um seguido do nome, até o fim da linha. A intuição é boa: basta procurar esse padrão em cada texto e guardar o que vem depois.

A ideia inicial seria coletar os dois. Antes de fazer isso, uma pergunta.

O que identifica uma pessoa

Insper

Coletar os dois?

Vocês acabaram de ler o nome de quem pediu o canabidiol e, no corpo da sentença, a doença que ela tem.

Se o nosso CSV ganhar a coluna `requerente` e for parar no repositório do grupo, no GitHub, o que acontece? Vocês veem algum problema?

Dado pessoal, e de saúde

RESPOSTA

Lei Geral de Proteção de Dados. Art. 5º, I.

> I - dado pessoal: informação relacionada a pessoa natural identificada ou identificável;

A sentença é pública, e isso não torna o nome junto da doença um dado livre para qualquer uso. Para o cliente, quem pede não importa: importa contra quem se pede. Hoje coletamos só o `requerido`.

Basta procurar o padrão

Expressão regular: um padrão escrito em código

Expressão regular (regex) é uma linguagem pequena, dentro do Python, para descrever um padrão de texto: "a palavra Requerido, dois-pontos, e depois tudo até o fim da linha".

Ela tem uma gramática própria, cheia de símbolos. Não vamos aprendê-la hoje: vamos pedir ao assistente do VS Code, que a escreve bem, e conferir o resultado.

Peçam ao assistente

No caderno, peçam ao assistente do VS Code, em palavras:

> "Uma expressão regular que pegue, em cada texto da coluna `texto`, o que vem depois de `Requerido:` até o fim da linha, e guarde numa coluna nova `requerido`."

Rodem o que ele devolver e contem quantas linhas ganharam valor.

A que eu recebi, com acerto de 269 de 300

RESPOSTA

```
decisoes["requerido"] = decisoes["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
print(decisoes["requerido"].notna().sum())
print(decisoes["requerido"][:5])
```

Ler o padrão que o assistente escreveu

Pedaço	O que diz
`Requerid[oa]s?:`	Requerido, Requerida, Requeridos ou Requeridas, e os dois-pontos
`\s*`	espaços ou tabulação, quantos houver
`([^\n]+)`	tudo até a quebra de linha; os parênteses marcam o que guardar

Não é para escrever: é para conferir se o que o assistente entendeu é o que vocês pediram.

31 ficaram vazias

```
269
0       Fazenda Pública do Estado de São Paulo e outro
1    UNIMED DE GUARULHOS COOPERATIVA DE TRABALHO ME...
2    Fazenda Pública do Estado de São Paulo e INSTI...
3                                                  NaN
4                              UNIMED NACIONAL e outro
Name: requerido, dtype: str
```

`str.extract` guarda o trecho que corresponde ao padrão; onde não há, fica `NaN`, o vazio. As 31 não usam `Requerido:` (`Impetrado`, `Executado`, sem cabeçalho).

Dá para reduzir as 31?

Peçam ao assistente uma segunda rodada: "31 ficaram vazias; olhe os textos sem valor e amplie o padrão para os outros rótulos do polo passivo". Rodem e contem de novo.

Quantas sobram? E por que a regra sempre vai deixar alguma?

288 de 300, e as 12 que sobram

RESPOSTA

```
padrao = r"(?:Requerid|Impetrad|Executad|Reclamad)[oa]s?:
\s*([^\n]+)"
decisoes["requerido"]
= decisoes["texto"].str.extract(padrao)
print(decisoes["requerido"].notna().sum())
```

Impetrado, Executado e Reclamado entram; sobram 12 sem cabeçalho nenhum. O padrão cresce a cada rodada, e sempre há um texto que ele não descreve.

Quem é mais acionado?

Com a coluna `requerido` pronta, `value_counts` conta quantas vezes cada valor aparece:

```
import pandas as pd

decisoes = pd.read_csv("decisoes-canabidiol-tjsp.csv")
decisoes["requerido"] = decisoes["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
print(decisoes["requerido"].value_counts().head(6))
```

Olhem a lista que sai. Ela responde à pergunta 2 do cliente?

A mesma Fazenda, escrita de três jeitos

RESPOSTA

```
requerido
Fazenda Pública do Estado de São Paulo             66
Fazenda Pública do Estado de São Paulo e outro     28
Sul América Companhia de Seguro Saúde               9
PREFEITURA MUNICIPAL DE JABOTICABAL e outro          8
PREFEITURA MUNICIPAL DE SÃO PAULO                    4
PREFEITURA MUNICIPAL DE LIMEIRA e outro              4
Name: count, dtype: int64
```

Contar ainda não é responder

O que a lista mostra	O que falta
`Fazenda Pública do Estado de São Paulo` e a mesma `... e outro`	juntar: é o mesmo réu
`PREFEITURA MUNICIPAL DE X` em caixa alta	padronizar a grafia
`e outro`, `e outros`	separar o segundo réu

Padronizar é a aula de limpar uma base. Hoje a resposta ao cliente vem com essa ressalva escrita.

Público ou privado

Insper

Como dividiriam?

Temos a coluna `requerido`, com Fazenda Pública, prefeituras, Sul América, Unimed, Amil, Bradesco Saúde. Como vocês dividiriam esses valores entre público e privado, numa regra que o programa aplique às 300 linhas?

Uma regra por palavras

RESPOSTA

```
decisoes["requerido"] = decisoes["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
publico = ("Fazenda|Estado|Munic|Prefeitura"
           "|União|Secretaria|SUS")
decisoes["publico"] = decisoes["requerido"].str.contains(
    publico, case=False)
print(decisoes["publico"].value_counts(dropna=False))
```

O que a regra diz

A barra `|` é "ou": basta uma das palavras aparecer. `Munic` pega Município e Municipal. `case=False` ignora maiúsculas. `dropna=False` conta também as linhas vazias.

187 públicos, e o resto não é tudo privado

```
publico
True     187
False    113
Name: count, dtype: int64
```

`str.contains` testa se alguma das palavras aparece no requerido, sem distinguir maiúsculas. Entre as 300: 187 com ente público; 82 sem nenhuma das palavras, que a regra chama de privado; e as 31 sem `Requerido:`, que caem no `False` junto com elas.

Primeira resposta ao cliente, com ressalva

Nesta base, mais de dois terços das ações são contra o Estado ou um município.

A ressalva é o termo de busca: "canabidiol" puxa quem pede ao SUS. Contra um plano de saúde, talvez se escreva de outro jeito, e essas sentenças ficaram fora da coleta.

Quem ganha

Procurar "procedente"

```
import pandas as pd

decisoes = pd.read_csv("decisoes-canabidiol-tjsp.csv")
print(decisoes["texto"].str.contains("procedente").sum())
print(decisoes["texto"].str.contains("improcedente").sum())
```

```
165
68
```

165 procedentes e 68 improcedentes? O que está errado nessa conta?

165 inclui os 68

RESPOSTA

"improcedente" contém "procedente": toda sentença que diz "improcedente" entra também na conta de "procedente". Os 165 incluem os 68. E "PROCEDENTE" em caixa alta escapa das duas contas.

Procurar uma palavra solta não responde "quem ganhou". O padrão precisa dizer o que vem antes e depois dela.

A melhor regex que o assistente conseguiu

Pedi: "a melhor expressão regular possível para dizer se a sentença julgou procedente, parcialmente procedente, improcedente ou extinguiu sem mérito". Voltou isto, em quatro grupos. É para olhar, não para decifrar:

```
PADRAO = re.compile(r"""
    (?P<parcial>
        julgo\s+(?:\w+\s+){0,3}?
        (?:parcialmente\s+procedentes?
          |procedentes?\s+em\s+parte)
      | parcial\s+procedência)
```

... o grupo improcedente ...

```
  | (?P<improcedente>julgo\s+(?:\w+\s+){0,3}?improcedentes?
      | denego\s+a\s+(?:segurança|ordem)
      | rejeito\s+o\s+pedido
      | nego\s+provimento)
```

... o grupo procedente ...

```
    | (?P<procedente>julgo\s+(?:\w+\s+){0,3}?procedentes?
          (?!\s+em\s+parte)
        | concedo\s+a\s+(?:segurança|ordem)
        | acolho\s+o\s+pedido)
```

... e o grupo extinto

```
   | (?P<extinto>julgo\s+extint[oa]s?\b[^.]{0,160}?
        sem\s+(?:resolução|julgamento|exame|análise)
        \s+d[oe]\s+mérito
      | extingo\s+o\s+(?:processo|feito)[^.]{0,160}?
        sem\s+(?:resolução|julgamento)\s+d[oe]\s+mérito
      | art(?:igo|\.)?\s*485
      | homologo\s+(?:a\s+desistência|o\s+acordo
                    |a\s+transação))
""", re.IGNORECASE | re.VERBOSE)
```

A função que aplica o padrão

```
def resultado(texto):
    achados = list(PADRAO.finditer(texto[-3000:]))
    if not achados:
        return "?"
    return achados[-1].lastgroup

decisoes["resultado"] = decisoes["texto"].apply(resultado)
print(decisoes["resultado"].value_counts())
```

Procura só no fim do texto, onde fica o dispositivo, e fica com a última frase que casou. Sem frase nenhuma, "?".

40 que ela não alcança

RESPOSTA

```
resultado
procedente      130
improcedente     88
?                40
parcial          27
extinto          15
Name: count, dtype: int64
```

Quatro grupos de frases, e 40 de 300 sem classe.

Complexa, e ainda assim incompleta

O que a regex faz	O que escapa
procura nos últimos 3.000 caracteres	dispositivo mais acima
quatro grupos de frases: julgo, concedo, denego, extingo	juiz que escreve de outro jeito
fica com a última frase que casou	sentença que julga dois pedidos

A regra acerta onde o texto segue o padrão que eu imaginei. Para saber quantas estão certas, alguém lê uma amostra.

Outra maneira: utilizando Inteligência Artificial

Insper

Quem mais poderia preencher as três colunas?

Em vez de descrever o padrão do texto, dá para pedir a alguém que leia cada sentença e responda: quem é o réu, se é público ou privado, qual foi o resultado.

Ler 300 à mão leva um dia. Quem mais lê texto e responde em formato fixo?

Um modelo de linguagem

RESPOSTA

Um modelo de linguagem lê a sentença e preenche o formulário. Chamado de dentro do programa, ele faz isso para cada linha da tabela. Hoje vamos só usar; como ele funciona por dentro e o que custa ficam para as próximas aulas.

Para que o modelo de linguagem faça isso, precisamos de três coisas: uma chave de acesso a um serviço que roda o modelo, um formulário com as perguntas que queremos fazer, e a biblioteca que liga os dois à tabela.

Criar a conta e a chave na Groq

#	O que fazer
1	Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail
2	No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1`
3	Copiar a chave agora: ela começa com `gsk_`

Hoje a chave vai colada no programa, e isso não é seguro: como guardá-la fora do código é assunto de uma aula adiante. Não mandem a chave a ninguém, nem para mim.

Ler o README de uma biblioteca

A biblioteca é o dataframeit, no GitHub. O README de uma biblioteca costuma ter o que basta para a primeira chamada: como instalar, um exemplo mínimo e a lista de argumentos.

```
uv add "dataframeit[groq]"
```

Vamos ler juntos o exemplo do README e adaptá-lo às nossas três perguntas.

O formulário

```
from typing import Literal
from pydantic import BaseModel
class Formulario(BaseModel):
    requerido: str
    polo_passivo: Literal["público", "privado"]
    resultado: Literal["procedente",
                       "parcialmente procedente",
                       "improcedente",
                       "extinto sem julgamento do mérito"]
```

O que cada linha da classe diz

Um campo por linha: nome, dois-pontos e tipo. `Literal` fecha a lista de valores que o modelo pode devolver; `str` é texto livre. A classe herda de `BaseModel` o trabalho de conferir cada resposta.

Uma chamada para dez sentenças

```
from dataframeit import dataframeit
dez = pd.read_csv("dez-decisoes.csv")
tabela = dataframeit(dez, Formulario,
    "Leia a sentença e preencha os campos: {texto}",
    provider="groq", text_column="texto",
    api_key="gsk_...")
print(tabela[["requerido", "polo_passivo", "resultado"]])
```

`dez-decisoes.csv`, no Blackboard, tem dez das 300. Uma linha de cada vez: o texto vai a um servidor da Groq, o modelo responde, a classe confere. O caderno grava `tabela` em `dez-decisoes-modelo.csv`.

Os argumentos do dataframeit

Argumento	O que é
`dez`	a base, uma sentença por linha
`Formulario`	o formulário, que confere cada resposta
o texto com `{texto}`	o pedido ao modelo; cada sentença entra no lugar de `{texto}`
`provider="groq"`	o serviço que roda o modelo
`text_column="texto"`	a coluna em que está o texto
`api_key="gsk_..."`	a sua chave, colada hoje no programa

Rodem nas dez

Com a chave no lugar de `gsk_...`, rodem a célula. Leva alguns segundos por sentença.

Deu certo quando `tabela` tem dez linhas e três colunas novas, cada uma com um valor que a classe aceitou. Comparem com o que a regex deu para as mesmas dez.

Os dois caminhos lado a lado

Guardar antes de comparar: `to_csv`

```
import pandas as pd

dez = pd.read_csv("dez-decisoes.csv")
dez["requerido_regex"] = dez["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
dez.to_csv("dez-decisoes-regex.csv", index=False)
```

`to_csv` grava a tabela num arquivo; `index=False` deixa de fora a numeração das linhas. O caderno gravou `tabela`, a do modelo, do mesmo jeito. Com os dois arquivos salvos, a comparação não depende de rodar tudo de novo.

As mesmas dez, regex contra modelo

```
import pandas as pd

regex = pd.read_csv("dez-decisoes-regex.csv")
modelo = pd.read_csv("dez-decisoes-modelo.csv")

regex["requerido_modelo"] = modelo["requerido"]
print(regex[["requerido_regex", "requerido_modelo"]])
```

`read_csv` relê os dois arquivos. Onde as duas colunas diferem, quem acertou?

Quando usar cada um?

Nas dez sentenças, comparem os resultados e discutam em dupla:

#	Pergunta
1	Em que tarefa a regex basta, e por quê?
2	Em que tarefa o modelo é melhor, e o que ele custa?
3	Como vocês saberiam quantas respostas do modelo estão certas?

Regra para o que tem forma, modelo para o que tem sentido

RESPOSTA

	Expressão regular	Modelo de linguagem
acerta	em texto de forma fixa	em texto variado
custa	testar a regra	uma chamada por linha
erra	em silêncio	com confiança
se confere	lendo o vazio	lendo uma amostra

O que ainda vamos aprender

Etapa	A pergunta que ela responde
definir a amostra	que termo, classe, período e tribunal, e por quê
o que coletar e o que não	que dado de pessoa fica de fora, e com que base
coletar	raspar e guardar, com a ficha do que foi feito
limpar	a mesma Fazenda com um nome só; vazios e duplicados
descrever	contar, agrupar e cruzar: público contra privado, quem ganha em cada grupo
visualizar	o gráfico que o cliente lê sem a tabela

Na próxima aula

A Temis, equipe vencedora da fase 1 da APS, conta como venceu. Depois, a etapa que hoje veio pronta: como escolher termo, classe, período e tribunal para coletar uma base que responda à pergunta.

Até o início dela, terça, 03/11: o caderno de hoje rodando no seu computador, com a tabela do modelo nas dez sentenças.